Facial Recognition Attendance Management System

Complete guide to your **project (FaceAttend Ultra, final version): brief, detailed, simple, with code, definitions, examples and viva answers**

Version 2 (final). This guide now covers the final **FaceAttend Ultra** version. **Sources read:** the Python files uploaded earlier (`main.py` … `test_system.py`; these are an **earlier build**), the Technology Stack PDF, your three text files (`faceAttend part 1-3`), the **Final Project Report (PDF)** and the **PowerPoint (slides + speaker notes)**. **Not uploaded:** `database/db_manager.py`, `security.py`, `email_notifier.py` and the final `face_recognition.py` (liveness code). Final-version features are explained from the report, PPT and text files, and the code shown for them is an **illustrative sketch** (marked as such), not your actual code. **Rule used when sources disagree:** the **final report wins** (for example the default threshold is 78, not 82 or 75).

PART A: THE BRIEF VERSION

A1. Explain the project in 30 seconds

FaceAttend Ultra is a **Python desktop application** that marks attendance from a **laptop webcam**, with no GPU and no extra hardware.

1. The administrator **logs in** (password checked against a salted hash) and reaches the dashboard.
2. A student is **enrolled**: details are saved and about **100 face photos** are captured (cropped, 200x200, contrast-equalised).
3. **Train Data** teaches the LBPH model using **only students who are still enrolled**.
4. In **live recognition**, each frame is halved and made grayscale, Haar finds faces, a shape filter removes odd boxes, LBPH gives a label and a distance, and **6 frames vote** under a threshold (default **78**).
5. A **liveness challenge** follows: two random actions (turn, look up, blink) within 12 seconds. If anything is unclear, the check fails and nobody is marked.
6. Attendance is saved in **SQLite once per student per day**, then written to a **CSV**, and an optional **e-mail goes to the parent** through Brevo.
7. The administrator can **review, edit, import and export** the records.

A2. The project in a table

Question	Answer
Language	Python 3.10+
Interface	Tkinter / ttk desktop windows
Face detection	OpenCV **Haar Cascade** (on a half-size grayscale frame)
Face recognition	OpenCV **LBPH** (radius 2, 8 neighbours, 8x8 grid, 200x200 faces)
Decision	Distance threshold (70 strict / **78 default** / 82 sensitive) + 6-frame voting + liveness challenge
Database	**SQLite** in WAL mode: `users`, `students`, `attendance`, `email_notifications`
Other storage	CSV files, `id_map.json`, face images in `data/`, model `classifier.xml`
Security	Salted **PBKDF2-HMAC-SHA256** hashes, constant-time comparison, bound SQL parameters
E-mail	**Brevo** Transactional Email API (optional, needs internet)
Test data	6 students, 615 images, 51 attendance rows, 77.5 MB model
Tests	38 automated tests in 5 suites (reported passing)

A3. Problem and objectives

Problem: Manual attendance wastes class time, allows proxy attendance (a friend answers for you), and paper records get lost or altered. **Objectives:** automate attendance with face recognition; provide secure login; manage student records; store attendance in a database and CSV; allow correction, import and export.

A4. Team (from `developer.py`)

Member	Module
Ved Prakash Pandey	Face detection module
Siddharth Singh	Student management
Deepak Sah	System testing
Santosh Sah	Attendance system
Anshudeep Shah	Login and security
Risan Shrestha	UI and dashboard

Guide: Mr. Rakesh Kumar Rai.

PART B: THE DETAILED VERSION

B1. Architecture and flow



B2. Folder and file map

Item	Purpose
`main.py`	Entry point; run this to start
`login.py`, `register.py`, `forget_password.py`	Authentication windows
`dashboard.py`	Main hub with 8 module buttons
`student.py`	Student CRUD + 100-photo capture
`train.py`	Builds the LBPH model from photos
`face_recognition.py`	Live recognition + auto attendance
`attendance.py`	Attendance table, update, CSV import/export
`developer.py`, `help_desk.py`	Information windows
`test_system.py`	Automated unit tests
`database/db_manager.py`	All database functions (not uploaded)
`database/attendance_system.db`	SQLite database file (per README)
`data/`	Face photos named `user.<student_id>.<count>.jpg`
`classifier/`	`classifier.xml` (model) and `id_map.json` (label map)
`attendance_records/`	`attendance.csv` and exported CSVs
`assets/`	Images for the GUI
`haarcascade_frontalface_default.xml`	Ready-made face detector
`.venv`, `__pycache__`	Virtual environment and cache (ignore)

B3. Libraries (`requirements.txt`)

Library	What it is	Used for
`opencv-contrib-python`	Computer vision library **with extra modules**	Camera, Haar detection, LBPH (`cv2.face`). Plain `opencv-python` will **not** work
`pillow`	Image library	Showing PNG/JPG in Tkinter
`numpy`	Fast array maths	Holding image data and labels
`pandas`	Data-analysis library	Listed in `requirements.txt`; the final report does not mention it, so check whether your code uses it
`mysql-connector-python`	MySQL driver	Optional MySQL support (SQLite is the default)
built-in: `tkinter`, `sqlite3`, `csv`, `json`, `glob`, `threading`, `unittest`	Standard library	GUI, DB, files, testing

---

PART C: MODULE-BY-MODULE WALKTHROUGH (WITH CODE)

C1. `main.py`: starting the app

```python
initialize_database()                                          # create tables if
os.makedirs(os.path.join(BASE_DIR, "data"), exist_ok=True)     # create folders
...
root = tk.Tk()
app = LoginWindow(root)
root.mainloop()
```

1. Creates the database tables.
2. Creates `data`, `classifier`, `attendance_records` and `assets` if missing (`exist_ok=True` means "no error if it already exists").
3. If the Haar XML is missing or smaller than 10,000 bytes, downloads it from OpenCV's GitHub.
4. Generates the GUI images if needed.
5. Opens the login window; `mainloop()` keeps the window alive and listening for clicks.

> Final version: according to the report, a **first-run setup** guides creation of the first administrator account.

C2. `login.py` : login portal

- `BiometricAnimationHUD` : an animated canvas: a face drawn with 34 dots (`base_nodes`) joined by lines (`connections`), a rotating ring and a moving laser line. `canvas.after(35, self.animate)` redraws it every 35 ms, which creates the animation.
- `LoginWindow.handle_login()` calls `verify_login(username, password)` . On success, `open_dashboard()` destroys the login widgets and builds `MainDashboard` **on the same window** (only one `Tk()` root is used).
- **Remember Me**: in the final version only the **username** is saved (`database/.user_session.json`) and refilled next time; the password is never stored. Login compares a salted PBKDF2 hash (see C15).
- Links: New User Register and Forget Password.

C3. `register.py` : new user

A split screen (banner + form). `handle_register()` validates:

```python
if not fname or not contact or not email or not sec_a or not password: ...  # r
if sec_q == "Select": ...                                                   # q
if password != confirm_pass: ...                                            # p
if not self.var_terms.get(): ...                                           # t
success, msg = register_user(fname, lname, contact, email, sec_q, sec_a, passwo
```

`tk.StringVar` variables are linked to the text boxes so `.get()` reads what the user typed.

C4. `forget_password.py` : password recovery

User selects a **security question**, types the **answer** and a **new password** (min 4 characters); `reset_password(question, answer, new_pass)` updates it. `grab_set()` makes the popup **modal** (must be closed first).

C5. `dashboard.py` : the hub

- Header, **live clock** (`root.after(1000, self.update_clock)` repeats each second), title strip, wallpaper canvas.
- **8 tiles in a loop** from `tiles_info` ; position from `row = idx // 4` , `col = idx % 4` . If an image is missing, a blue text button is used (fallback).
- Each opener creates a child window:

```python
def open_attendance(self):
    win = tk.Toplevel(self.root)
    AttendanceManagementSystem(win)
```

- `os.startfile(DATA_DIR)` opens the photos folder (Windows-only). `handle_exit()` asks confirmation.

C6. `student.py`: student management and photo capture (676 lines)

Screen: left = Current Course Information (Department, Course, Year, Semester) + Class Student Information (ID, Name, Division, Roll, Gender, DOB, Email, Phone, Address, Teacher, photo radio button) + buttons; right = search box and student table.

Buttons: SAVE (`add_student`), UPDATE (`update_student`), DELETE (`delete_student` and also removes the student's photos), RESET, ADD/UPDATE PHOTO SAMPLE.

Delete also cleans photos:

```python
old_samples = glob.glob(os.path.join(DATA_DIR, f"user.{sid}.*.jpg"))
for f in old_samples: os.remove(f)
```

`glob` finds files matching a pattern (`*` = anything).

`handle_take_photo_sample()`: how the 100 photos are captured

```python
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face_classifier.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5,
for (x, y, w, h) in faces:
    sample_count += 1
    face_raw  = gray[y:y+h, x:x+w]                                  # crop the face
    face_crop = cv2.equalizeHist(cv2.resize(face_raw, (200, 200)))   # 200x200
    cv2.imwrite(os.path.join(DATA_DIR, f"user.{student_id}.{sample_count}.jpg")
```

Steps: (1) check ID and name are filled, (2) save the student record first, (3) load the Haar file, (4) open webcam, (5) for each detected face crop, resize to 200x200, equalise, save as `user.<id>.<n>.jpg`, (6) stop at **100 samples** or on a key press (q, x, Esc, Enter, Space), (7) mark `photo_sample = Yes`, (8) **auto-train** by calling `train_classifier_core()`, (9) show "Generating datasets completed!!!".

Why the file name matters: `user.S101.7.jpg` splits by `.` into `user`, `S101`, `7`, `jpg`, so `train.py` can read the student ID from the file name. It's a tiny labelling system.

> **Final version:** the report gives the capture detector as `scaleFactor=1.1, minNeighbors=4, minSize=(40, 40)`. The code above is from the earlier build you uploaded (`1.3 / 5 / 60x60`). Use the report's values in the viva.

C7. `train.py`: learning the faces

Core function `train_classifier_core()`:

```python
image_paths = glob.glob(os.path.join(DATA_DIR, "*.jpg"))
for img_path in image_paths:
    parts = os.path.split(img_path)[1].split(".")       # ['user','S101','7','j
    raw_id = parts[1]                                    # 'S101'
    if raw_id not in id_to_int:                          # give each student a
        id_to_int[raw_id] = next_int_id;  int_to_id[str(next_int_id)] = raw_id;
    img_gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    img_eq   = cv2.equalizeHist(cv2.resize(img_gray, (200, 200)))
    faces.append(img_eq);  ids.append(id_to_int[raw_id])

clf = cv2.face.LBPHFaceRecognizer_create(radius=2, neighbors=8, grid_x=8, grid_
clf.train(faces, np.array(ids))
clf.write("classifier/classifier.xml")
json.dump({"id_to_int": id_to_int, "int_to_id": int_to_id}, ...)   # id_map.jso
```

- **Why the ID map?** LBPH only accepts **integer** labels, but student IDs can be text like `S101`. So `S101` becomes `1`, and `id_map.json` converts back during recognition.
- The function **returns** `(success, count, counts, message)` so the GUI and the tests can both use it.

`TrainDataWindow`: Clicking **TRAIN DATA** disables the button and starts a **background thread** (`threading.Thread(..., daemon=True)`) so the window doesn't freeze. The worker previews 12 samples per student with an on-screen progress bar (OpenCV window "Training"), calls `train_classifier_core()`, then shows a popup with a per-student breakdown. `root.after(0, ...)` is used to update the GUI from the thread.

> **Final version:** training uses **only students who are still in the database**; if no valid samples remain, the old model is **deleted**. Training also runs with radius 2, 8 neighbours, 8x8 grid.

C8. `face_recognition.py`: live recognition and attendance (the heart)

1. Load model and map

```python
clf = cv2.face.LBPHFaceRecognizer_create(radius=2, neighbors=8, grid_x=8, grid_
clf.read(classifier_path)                    # classifier.xml
int_to_id = json.load(f)["int_to_id"]        # label number -> student ID
```

2. Open camera: `cv2.VideoCapture(0, cv2.CAP_DSHOW)` (`0` = default camera; DirectShow = fast start on Windows; falls back to normal).

3. For every frame

```python
```

```
small_frame = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)          # half size =
small_gray  = cv2.cvtColor(small_frame, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(small_gray, scaleFactor=1.12, minNeighbor
```

- `scaleFactor=1.12` : image shrinks 12% per pass so both big and small faces are found.
- `minNeighbors=4` : a face is accepted only if 4 nearby windows agree (fewer false alarms).
- `minSize=(25,25)` : ignore tiny detections.

4. Prepare and predict

python

```
x, y, w, h = sx*2, sy*2, sw*2, sh*2                    # back to full-size co
face_crop = cv2.equalizeHist(cv2.resize(gray_full[y:y+h, x:x+w], (200, 200)))
label_int, distance = clf.predict(face_crop)
raw_student_id = int_to_id.get(str(label_int))
if distance <= 82 and raw_student_id: ...
```

`predict` gives the best label and a **distance** (0 = identical; higher = less similar). At most **82** is accepted.

5. Show and log

- Student details from the DB via `get_student_by_id` , **cached** in `student_cache` (no DB query every frame).
- **Green box** + ID, Roll, Name, Department for known; **red box** "Unknown Face" otherwise.
- Attendance once per session:

python

```
if s_id not in self.logged_session_students:
    log_attendance(s_id, roll, name, dep, time_str, date_str, "Present")   # da
    log_attendance_to_csv(s_id, roll, name, dep, time_str, date_str, "Present")
    self.logged_session_students.add(s_id)
```

- `log_attendance_to_csv` scans the CSV; if the same **roll** + **date** already exists it doesn't write again. The CSV has columns `Attendance ID, Roll, Name, Department, Time, Date, Attendance` ; the ID is the row number.

6. Exit: keys q, x, Esc, Enter, Space or close the window; then `cap.release()` frees the camera.

> **Final version of recognition:** the default threshold is **78** (70 / 78 / 82 selectable), live Haar uses `scaleFactor=1.15, minNeighbors=6` , boxes must pass the 0.70 to 1.35 shape filter, **6 frames vote**, then a **liveness challenge** runs, and only then is attendance saved (database

> first, then CSV, then the parent e-mail). The code above is the earlier build (`distance <= 82`, no voting or liveness). See C13 to C16.

C9. `attendance.py` : managing records

Function	Purpose
`load_attendance_from_csv()`	Reads `attendance.csv` into the table (falls back to the DB if no CSV)
`handle_row_selection()`	Click a row to fill the form
`handle_update()`	Change Present/Absent in **DB and CSV**
`handle_import_csv()`	File dialog, reads 7-column rows, calls `log_attendance` for each
`handle_export_csv()`	Saves the table as `attendance_export_YYYYMMDD_HHMMSS.csv`
`handle_reset()`	Clears the form

`ttk.Treeview` = the table; `filedialog` = Open/Save dialogs; `messagebox` = popups.

C10. `help_desk.py` , `developer.py`

Help Desk shows a clickable email (`webbrowser.open("mailto:...")`). Developer builds 6 profile cards using a single reusable function `create_dev_card()` fed from the `DEVELOPERS` list (add a member = add one dictionary).

C11. `db_manager.py` : the database layer (file not uploaded; confirmed from the final report and PPT)

Functions used by the other files: `initialize_database`, `register_user`, `verify_login`, `reset_password`, `add_student`, `update_student`, `delete_student`, `fetch_all_students`, `search_students`, `get_student_by_id`, `update_photo_sample_status`, `log_attendance`, `fetch_all_attendance`, `update_attendance_record`.

Four tables (from the final report)

Table	Main fields	Rule
`users`	id, fname, lname, contact, email (unique), security_q, security_a (hash), password (hash), created_at	Administrator login and recovery
`students`	student_id (primary key), dep, course, year, semester, division, name, roll, gender, dob, email, phone, address, teacher, photo_sample, **parent_email** (16 fields)	Student registry
`attendance`	attendance_id, student_id, roll, name, department, time, date (DD/MM/YYYY), status	**Unique index on student_id + date**
`email_notifications`	student, roll, attendance_date, status, recipient_email, delivery_result, provider_message_id, error_details, time	**Unique on student + date + status**

Key ideas

- **Primary key:** a column whose value is unique for every row (for example `student_id`).
- **Unique index:** the database itself refuses a second row with the same combination.

```sql
CREATE UNIQUE INDEX IF NOT EXISTS ux_att_student_date ON attendance(student_id,
```

- **WAL mode:** `PRAGMA journal_mode = WAL` lets one window read while recognition writes.
- Most functions return `(success, message)` so the GUI can show the message.
- Deleting a student also removes related rows (cascade deletion, covered by the security-and-integrity tests).

C12. `test_system.py`: automated testing (`unittest`)

Test	What it checks
`test_01_user_auth_and_recovery`	Register, login, wrong password rejected, reset password, login with new password
`test_02_student_crud`	Add, fetch, update, search, photo-status update, delete a student
`test_03_training_and_lbph_classifier`	`train_classifier_core()` succeeds; `classifier.xml` and `id_map.json` exist
`test_04_attendance_logging`	Logs attendance, fetches records, updates a record
`test_05_gui_module_imports`	Every window class imports without error

`setUpClass` runs once before all tests (prepares DB/folders). Test names start with `test_` and numbers keep them in order. Run: `python test_system.py`.

C13. Final decision logic: threshold, voting and shape filter (from the final report; code is an illustrative sketch)

Setting	Value in the final report
Capture detector (Haar)	`scaleFactor=1.1`, `minNeighbors=4`, `minSize=(40, 40)`
Live detector (Haar, on half-size grayscale)	`scaleFactor=1.15`, `minNeighbors=6`
Shape filter	Keep boxes whose width / height is between **0.70 and 1.35**
LBPH	radius 2, 8 neighbours, 8x8 grid, 200x200 faces
Thresholds	**70 strict, 78 balanced (default), 82 more sensitive**
Immediate accept	Distance **50 or less**
Tracker memory	Last **6 predictions** per tracked face
On-screen percentage	A rough similarity value, **not a probability**

```python
```

```python
# ILLUSTRATIVE SKETCH of the idea (not your exact code)
from collections import deque, Counter
history = {}                                          # one short memory per tr

def decide(face_id, label, distance, threshold=78, min_votes=4):
    q = history.setdefault(face_id, deque(maxlen=6))  # keeps only the last 6
    q.append(label if distance <= threshold else "Unknown")
    if distance <= 50:                                # very close match: accep
        return label
    winner, votes = Counter(q).most_common(1)[0]
    if winner != "Unknown" and votes >= min_votes:    # enough frames agree
        return winner
    return None                                       # not confirmed yet
```

> The report does not state the exact number of agreeing frames (`min_votes`). Check your code so you can answer that in the viva.

C14. Liveness challenge (final version; sketch is illustrative)

Purpose: a printed photo cannot turn its head or blink, so a random request makes a still picture fail.

1. Runs **only after** the LBPH match is confirmed.
2. Picks **2 random actions out of 8 presets** (turn the head, look up, return to centre, blink, and so on).
3. Is tied to the **tracked face**: it collects a short baseline, then watches how the facial features and face box move.
4. Time limits: **5.5 s per action, 12 s in total.**
5. A **blink** counts only if the eyes go **open, closed, open** within **0.10 to 1.20 s.**
6. **Fails closed:** if the face changes, is lost, time runs out or the reading is uncertain, the check fails, and a short **cooldown** follows before a retry.

python

```python
# ILLUSTRATIVE SKETCH of the blink rule
state, t_closed = "OPEN", 0.0
def on_eye_reading(eyes_open, now):
    global state, t_closed
    if state == "OPEN" and not eyes_open:
        state, t_closed = "CLOSED", now
    elif state == "CLOSED" and eyes_open:
        state = "OPEN"
        return 0.10 <= (now - t_closed) <= 1.20     # True = valid blink
    return False
```

Limits (stated in your report): it is software-only on a 2D webcam; it makes a still photo fail and a fixed recording less likely to match, but real photo/video attacks were **not tested**, and it cannot promise protection against advanced replay, live screen injection, 3D masks or generated video.

C15. Security (`security.py`) (final version; sketch is illustrative)

python

```python
import hashlib, hmac, os
ITER = 200_000                                          # example value; yours
def hash_password(password):
    salt = os.urandom(16)                               # random salt
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, ITER)
    return salt.hex() + "$" + digest.hex()              # store salt + hash, ne

def verify_password(password, stored):
    salt_hex, digest_hex = stored.split("$")
    test = hashlib.pbkdf2_hmac("sha256", password.encode(), bytes.fromhex(salt_
    return hmac.compare_digest(test.hex(), digest_hex)  # constant-time compari
```

- **Login:** read the stored salt and hash, hash the typed password the same way, compare in constant time.
- **Security answers** are hashed the same way.
- **Upgrade at start-up:** old plain-text values are converted to hashes.
- **Remember Me** stores **only the username**.
- **Bound parameters:** the SQL text and the values are sent separately.

python

```python
cur.execute("SELECT * FROM users WHERE email = ?", (email,))                # s
cur.execute("SELECT * FROM users WHERE email = '" + email + "'")            # u
```

- **Not protected:** face images and model files are **not encrypted**.

C16. Parent e-mail (`email_notifier.py`) (final version; sketch is illustrative)

Flow: Rahul is recognised, so attendance is saved in SQLite first. Then the parent's e-mail is read from the `students` table, a **background thread** sends the request to Brevo, and the attempt is stored in `email_notifications`.

python

```
# ILLUSTRATIVE SKETCH
import json, threading, urllib.request
def send_email(api_key, to_email, subject, html):
    body = json.dumps({"sender": {"name": "FaceAttend", "email": SENDER},
                       "to": [{"email": to_email}],
                       "subject": subject, "htmlContent": html}).encode()
    req = urllib.request.Request("https://api.brevo.com/v3/smtp/email", data=bo
                                 headers={"api-key": api_key, "content-type": "
    return urllib.request.urlopen(req, timeout=15).read()

def notify_async(*args):
    threading.Thread(target=send_email, args=args, daemon=True).start()   # cam
```

- Results are shown on screen with `root.after(0, ...)`, because Tkinter must be updated from its **main thread**.
- The **audit row** (student, roll, date, status, recipient, delivery result, provider message ID, error details, time) uses a **unique key on student + date + status**, so the same event is not logged twice.
- **Needs** internet, a valid Brevo API configuration and a valid recipient e-mail.

PART D: THE TWO CORE ALGORITHMS IN SIMPLE WORDS

D1. Haar Cascade: "Where is a face?"

- Looks for simple **light/dark rectangle patterns**: eyes are darker than cheeks; the nose bridge is brighter than the eye sockets.
- **Integral image** = a trick that lets it add up pixels in any rectangle very quickly. **AdaBoost** = combines many weak checks into one strong decision.
- **Cascade** = many stages. Non-face areas fail early, cheap stages and are discarded fast; only real faces pass every stage, which is why it is fast.
- The trained rules live in `haarcascade_frontalface_default.xml`. Works mainly for **front-facing** faces.

D2. LBPH: "Whose face is it?"

1. For each pixel, compare it to its 8 neighbours (`neighbors=8`, distance `radius=2`).
2. Neighbour >= centre gives **1**, otherwise **0**, forming an 8-bit number. **Example:** centre = 50; neighbours 60, 40, 55, 30, 70, 45, 50, 20 give `1 0 1 0 1 0 1 0` = binary 10101010 = **170**.
3. Doing this for all pixels gives a texture image that barely changes with brightness.
4. Split the face into an **8 x 8 grid** (`grid_x=8, grid_y=8`, 64 cells). Count how often each value appears in each cell (a **histogram**) and join all 64 histograms into one long signature.
5. To recognise, compute the live signature and compare with all stored ones; the **smallest distance** wins. Accepted if the distance is within the chosen threshold (70 / **78 default** / 82); 50 or less is accepted at once.

Why LBPH? Fast on any laptop, works with small datasets, easy to retrain.

PART E: GLOSSARY: BASIC TERMS WITH DEFINITIONS AND EXAMPLES

Term	Simple definition	Example in this project
GUI	Graphical User Interface	Dashboard with buttons
Tkinter / TTK	Python's built-in GUI toolkit / its themed widgets	`tk.Button`, `ttk.Treeview`
Widget	Any GUI element	Label, Entry, Combobox
root / mainloop()	Main window / loop that keeps it running	`root.mainloop()`
Toplevel	A child window of the main window	Attendance window
StringVar	Variable linked to a widget's text	`var_name`
Treeview	Table widget	Student and attendance tables
pack / grid / place	Layout methods (stack / rows-columns / exact position)	Tiles use `grid`
Modal window	Popup that must be closed first	Forget Password (`grab_set`)
after()	Run a function after N milliseconds	Clock, animation
Thread	A parallel path of execution so the GUI doesn't freeze	Training worker
daemon thread	Background thread that stops when the app closes	`daemon=True`
OpenCV (cv2)	Open-source computer vision library	Camera, detection, recognition
Frame	One still picture from a video	30 frames per second
Pixel	Smallest dot of an image	200x200 = 40,000 pixels
Grayscale	Image with only shades of grey	Faster than colour
BGR	OpenCV's colour order (Blue-Green-Red)	Differs from normal RGB

Term	Simple definition	Example in this project
Histogram	Count of how often each value occurs	LBPH signature
Histogram equalisation	Spreads brightness to improve contrast	`cv2.equalizeHist`
Face detection	Finding where faces are	Green/red rectangle
Face recognition	Deciding whose face it is	Shows student's name
Haar Cascade	Fast detector from light/dark patterns	The XML file
LBPH	Recognition method based on local texture	`LBPHFaceRecognizer_create`
Bounding box	Rectangle around a face `(x, y, w, h)`	`cv2.rectangle`
Dataset	Collection of training data	100 photos per student in `data/`
Training	Teaching the model from labelled data	Train Data
Model / classifier	The saved result of training	`classifier.xml`
Label	The number/name attached to a training image	1, 2, 3 mapped by `id_map.json`
Distance (confidence)	How different two faces are; lower = better	46.8 average enrolled (bright light), 81.4 or more unregistered
Threshold	Cut-off for accepting a match	78 (default)
False positive / negative	Wrongly accepting / wrongly rejecting	Look-alike accepted / genuine student shown Unknown
Database (SQLite)	Organised storage in tables, in one file	`attendance_system.db`
CRUD	Create, Read, Update, Delete	Student buttons
CSV	Comma-Separated Values text file, opens in Excel	`attendance.csv`

Term	Simple definition	Example in this project
JSON	Text format for structured data	`id_map.json`
Caching	Remembering results to avoid repeat work	`student_cache`
Unit test	Small automated check of one function	`test_system.py`
Virtual environment	Isolated set of packages for a project	`.venv`
Proxy attendance	Marking attendance for someone else	The problem we solve
Session	One run of the recogniser window	`logged_session_students`

PART F: HOW TO RUN AND DEMO

```bash
python -m venv .venv
.venv\Scripts\activate            # Windows
pip install -r requirements.txt
python main.py
```

Demo order (about 4 minutes)

1. **Login** (first-run setup creates the first administrator).
2. **Student Details:** fill the form (include the **parent e-mail**), Save, then take the photo sample until about 100 images are captured.
3. **Train Data** (only enrolled students are used).
4. **Face Recognition:** choose a threshold (78 is the default), look at the camera until the match is confirmed, then **do the two random actions** shown on screen.
5. Show the green box, the saved **attendance row**, and the parent e-mail (needs internet and the Brevo setup; otherwise show the `email_notifications` row).
6. **Attendance:** edit a status, **Export CSV**.
7. Close.

Tips: good front lighting; look straight at the camera and move your head slightly during capture; retrain after adding or deleting a student; use 70 in good light and 82 only in poor light; never show real passwords or API keys on screen.

PART G: VIVA QUESTION BANK WITH ANSWERS

(Merged and expanded: your old Part G and Part S, plus every question heading in your three text files. Each answer is short, then explained in simple words with an example.)

1. Project basics

Q1. What is FaceAttend Ultra and what problem does it solve? A desktop app that marks attendance automatically from a webcam. It removes slow roll calls, proxy attendance (a friend answering for you), and manual typing into Excel. Example: Rahul walks up, the camera recognises him, he does a quick blink, and he is marked Present.

Q2. Why face recognition and not RFID cards or fingerprints? Cards can be lent to a friend; fingerprint scanners need extra hardware and physical contact. Every laptop already has a webcam, so nothing new is bought.

Q3. Why Python, Tkinter, OpenCV and SQLite? Python has the libraries; Tkinter is built in for windows and buttons; OpenCV gives camera, Haar and LBPH; SQLite is a one-file database with no server.

Q4. Why not deep learning? Deep models usually need a GPU and lots of training data. This project targets an ordinary laptop, so it uses lighter classical methods and accepts lower robustness.

2. Image preparation

Q5. Why grayscale? LBPH uses brightness and texture, not colour. One channel instead of three is also faster.

Q6. What is cropping and why do it? Keeping only the face and dropping the classroom background. Example: store the 200x200 face, not the whole webcam picture.

Q7. Why resize to 200x200? Every image must have the same size in capture, training and recognition: 200 x 200 = 40,000 pixels.

Q8. What is histogram equalisation? It spreads brightness values to improve contrast. Example: a dim face in a dark room becomes clearer. It reduces, but does not remove, the effect of lighting.

Q9. Why is the same preparation used everywhere? LBPH compares patterns, so the new face must be prepared exactly like the training faces, otherwise distances become meaningless.

Q10. Why about 100 samples per student? Different frames show slightly different angles and expressions, so the model learns more variation.

3. Face detection (Haar Cascade)

Q11. How does Haar Cascade work? It checks light/dark rectangle patterns (eyes darker than cheeks, nose bridge brighter). An integral image makes each check fast, and a cascade of stages discards non-face areas early.

Q12. Why halve the frame before detection? A half-width, half-height image has 75% fewer pixels, so detection is faster (reported 11.8 to 13.2 ms per frame).

Q13. What do scaleFactor, minNeighbors and minSize mean? `scaleFactor`: how much the image shrinks per search pass (1.15 means divided by 1.15 each time, about 13% smaller).

`minNeighbors`: how many nearby detections must agree (higher = fewer false boxes). `minSize`: ignore smaller boxes.

Q14. Why the shape filter 0.70 to 1.35? A real face box is roughly as wide as it is tall. Example: 100x110 gives 0.91 (kept); 200x80 gives 2.5 (rejected).

Q15. What are Haar's limits? Works mainly on frontal faces; struggles with side views, strong shadows and masks.

4. Face recognition (LBPH)

Q16. What is LBPH? Local Binary Patterns Histograms. It answers "whose face is this?" by comparing local texture patterns.

Q17. How does LBPH work? For each pixel, compare 8 neighbours with the centre (1 if neighbour >= centre, else 0) to form an 8-bit number. Example: centre 50, neighbours 60,40,55,30,70,45,50,20 give 10101010 = 170. Split the face into 8x8 = 64 cells, make a 256-bin histogram per cell, and join them: 64 x 256 = **16,384 values**.

Q18. Why radius 2, 8 neighbours and an 8x8 grid? 8 neighbours gives an 8-bit code; radius 2 looks at slightly larger features; the 8x8 grid keeps spatial information (which part of the face the pattern came from).

Q19. What is the "distance"? How different the live face is from the closest trained face. **Lower = more similar.**

Q20. Why do we need `id_map.json`? LBPH accepts only integer labels, so `S101` becomes `1`. The JSON file converts the label back to the student ID.

5. The decision (threshold and voting)

Q21. Why do we need a threshold? The recognizer always returns the closest enrolled student, even for a stranger. The threshold rejects matches that are not close enough.

Q22. What do 70, 78 and 82 mean? 70 strict (may reject real students in poor light); **78 default** (between enrolled distances up to about 74 and unregistered from about 81.4); 82 sensitive (just reaches into the unregistered range, use only in poor light).

Q23. Why is a distance of 50 or less accepted at once? It is a very close match, so no waiting for more votes is needed.

Q24. What is multi-frame voting? The tracker keeps the last 6 predictions for each face. Example: Rahul, Rahul, Rahul, Rahul, Unknown, Rahul gives enough agreement, so Rahul is confirmed. One blurry frame cannot cause a wrong mark.

Q25. Is the on-screen percentage a probability? No, only a rough similarity value.

6. Training

Q26. Why train only on currently enrolled students? Suppose Rahul is deleted but his old photos remain in `data/`. If they were used, the new model could still recognise him. The program checks the database before training.

Q27. What if no valid samples remain? The old model is deleted instead of being kept.

Q28. Why retrain after adding a student? The model only knows faces it was trained on.

7. Liveness

Q29. What is liveness detection and why is it needed? A check that a real person is present, not a photo or a video. A printed photo cannot follow random instructions.

Q30. How does your challenge work? After the match, 2 random actions out of 8 presets; 5.5 s per action, 12 s total; features and face-box movement are measured against a short baseline.

Q31. What counts as a blink? Eyes open, closed, open again within 0.10 to 1.20 seconds.

Q32. What does "fail closed" mean? When unsure (face changes, face lost, time out), the result is **fail**, not pass. Safer than letting someone through by mistake.

Q33. Can it stop all spoofing attacks? No. It is software-only on a 2D webcam, real attacks were not tested, and advanced replay, screen injection, 3D masks or generated video are not guaranteed to be stopped. Depth or infrared sensors would be needed for high security.

Q34. Can a photo fool the system now? A still photo should fail the random-action step. A video clip is less likely to match the random request, but this is not proven.

8. Database and attendance

Q35. Why SQLite? No server, one file, built into Python, ACID-safe, enough for a desktop app.

Q36. What is WAL mode? Write-Ahead Logging: changes are logged first, so one window can read attendance while recognition writes a new record.

Q37. What is daily deduplication? Preventing duplicate records. Example: Rahul stands in front of the camera ten times; only one row exists for that date. A **unique index on student_id + date** makes the database refuse the second row.

Q38. Why write the database first and the CSV afterwards? The database is the source of truth; the CSV copy is appended **only if the database write succeeded**, so the two cannot disagree.

Q39. Why keep a CSV at all? Teachers can open it in Excel or Google Sheets.

9. Security

Q40. Why hash passwords? If someone copies the database, they must not read passwords. Only a derived hash and a salt are stored.

Q41. What is PBKDF2-HMAC-SHA256? PBKDF2 = Password-Based Key Derivation Function 2, which repeats the work many times to slow guessing. HMAC = a hash combined with a secret key. SHA-256 = a hash producing a 256-bit digest.

Q42. What is salt? Random data added before hashing, so two identical passwords have different stored results.

Q43. How does login work? Read the stored salt and hash, hash the typed password the same way, compare in constant time (this reduces timing leaks).

Q44. What does Remember Me store? Only the username, not the password.

Q45. What are bound parameters? SQL text and values are sent separately, which blocks SQL injection.

Q46. Are face images and the model encrypted? No. They are sensitive data, so consent, retention and deletion rules are needed.

10. Parent e-mail

Q47. How does the parent e-mail work? After attendance is saved, a background thread calls the Brevo REST API; the attempt is logged in `email_notifications`.

Q48. Why a background thread? If sending takes 3 seconds, the camera window would freeze. A thread lets the app keep working.

Q49. What is an audit record? A log row with student, date, status, recipient, delivery result, provider message ID, error details and time, so an administrator can see what succeeded or failed. A unique key on student + date + status prevents repeats.

Q50. What does the e-mail feature need? Internet, valid Brevo configuration, and a valid parent e-mail.

11. Software design

Q51. Why `Toplevel` and not another `Tk()`? Tkinter should have one root; Toplevel windows are its children.

Q52. Why cache student details? To avoid a database query on every frame.

Q53. Why do threads use `root.after(0, ...)`? Tkinter widgets must be updated from the main thread.

12. Testing and results

Q54. How did you test? 38 automated tests in 5 suites: liveness 9, security and integrity 8, core system 6, Brevo e-mail 11, parent e-mail automation 4, plus a manual run of the whole workflow.

Q55. What do the tests not prove? Real photo or screen attacks, real e-mail delivery, and recognition accuracy on a group of people (liveness tests use synthetic frames; e-mail tests use a mock).

Q56. Explain your results. Enrolled students: 41.2 to 55.4 (bright) and 66.5 to 73.8 (ambient). Unregistered: 81.4 to 89.2. Non-face background: above 92. The default 78 sits in the gap.

Q57. Your video rate is 52 to 60 FPS, but webcams give 30. Explain. A processing rate cannot exceed the camera's own rate, as the report notes. Say "measured on one laptop, limited by the camera, rough values".

Q58. How well does it scale? Not well. One histogram is stored per image: 16,384 x 8 bytes = about 126 KB per image, about 12.6 MB per student, about 3.8 GB for 300 students, and matching time grows to roughly 0.2 s per face.

13. Limitations and future work

Q59. Main limitations? Software-only liveness; small informal test with no false-accept/false-reject rates; sensitive to light and angle; model grows with students; password minimum is only 4; CSV import needs re-checking; images and models not encrypted; Windows-specific functions.

Q60. Future work? Real attack testing, larger test with error rates, stronger account handling, SQLite as the only source of attendance, an embedding model for large groups, consent and

retention rules, ERP link and web view for parents.

PART H: LIMITATIONS AND WHAT CHANGED IN THE FINAL VERSION

Mentioning these yourself shows understanding. "Status" is based on the final report, PPT and text files.

#	Earlier observation (uploaded code)	Status in the final version
1	"Remember Me" stored the password in plain text	**Fixed:** stores only the username
2	Passwords and security answers possibly unhashed	**Fixed:** salted PBKDF2-HMAC-SHA256 hashes; old plain text upgraded at start-up
3	Duplicate attendance possible	**Fixed:** unique index on student + date; CSV written only after the DB write succeeds
4	Deleted students could still be recognised	**Fixed:** training uses only enrolled students
5	A photo could mark attendance	**Improved:** random liveness challenge (not tested against real attacks)
6	Single-frame decision	**Fixed:** 6-frame voting and a selectable threshold
7	Password reset by question + answer only	**Still true:** future work is a one-time e-mail reset
8	Weak validation	**Still true:** minimum password length is only 4
9	Capture and recognition use different Haar settings	**By design:** 1.1 / 4 / 40x40 for capture, 1.15 / 6 for live detection
10	Retrains on all photos each time	**Still true:** model grows about 12.6 MB per student
11	Windows-only functions (`CAP_DSHOW` , `os.startfile`)	**Still true**
12	`face_recognition.py` name clashes with the popular library	Check and rename if the final code still uses it
13	Clock format `%H:%M:%S %p`	Check in the final code; use `%I:%M:%S %p` for 12-hour time

New limitations stated in the final report

- The liveness check is software-only; no measured attack success rate.

- The test setup was small and informal (one laptop, no false-accept/false-reject rates, thresholds not validated).
- Tests use synthetic frames and mocked services.
- CSV import must be re-checked (an earlier review found a column-layout mismatch with export).
- Failures when deleting face files or retraining may not always be shown to the user.
- Face images and models are not encrypted; consent and retention rules are needed.
- Some test scripts may still use the live database.

Future scope: see Q60 in Part G.

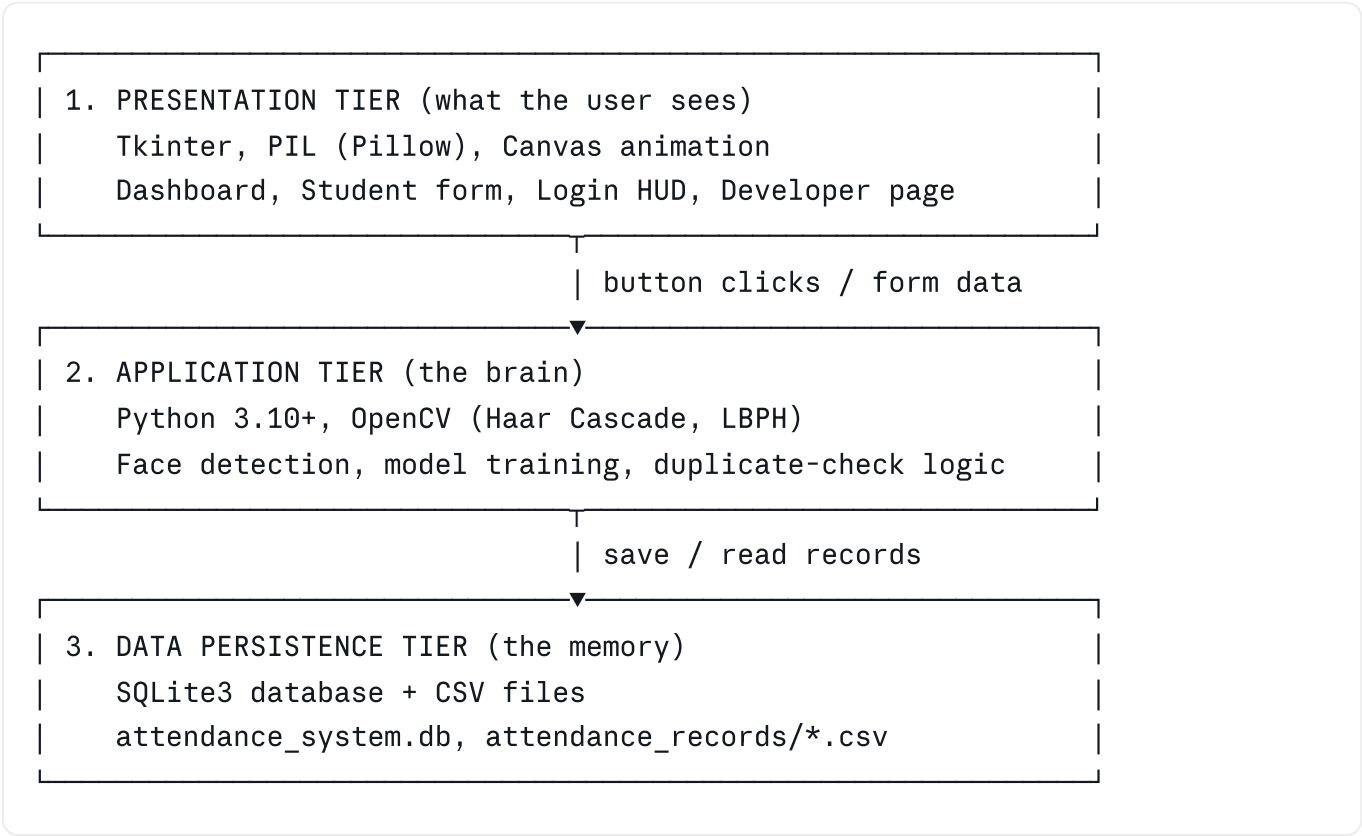
PART I: YOUR FINAL REPORT vs. SUGGESTED REPORT CHAPTERS

Your final report has	Guide part to use for extra depth
Abstract, Introduction	A1-A3
Dataset Description (4 tables)	C11, Part M
Methodology (pipeline, training, recognition, decision, liveness, security)	C7, C8, C13-C16, Part D
Results and Discussion	Part N
Comparison with Existing Work	Part O
Applications, Limitations, Future Work	Part H
Conclusion, References	Part Q

Often expected by examiners but missing from your report (add them if your department's format requires it): list of figures and tables, abbreviations list, hardware and software requirements (Part P), 3-tier architecture diagram (Part J), use-case diagram, data-flow diagram, ER diagram of the 4 tables, flowchart of the pipeline, **screenshots** of every window, and an appendix with key code snippets.

PART J: 3-TIER LAYERED ARCHITECTURE (added from the Technology Stack Report)

The project is organised in **three layers**. Each layer has one job and talks only to the layer next to it. This is called **separation of concerns**.



Tier	Main technologies	Your files	Role
1. Presentation (Frontend)	Tkinter, PIL, Canvas	`login.py`, `dashboard.py`, `student.py`, `attendance.py`, `developer.py`, `help_desk.py`	Windows, forms, live video overlay, form validation
2. Application (Backend / AI)	Python, OpenCV	`face_recognition.py`, `train.py`, capture logic in `student.py`	Real-time frame processing, feature extraction, matching, duplicate logic
3. Data Persistence (Database)	SQLite3, CSV	`database/db_manager.py`, `attendance_records/`	Student CRUD, attendance storage, CSV sync

> Final version: `security.py` (hashing) and `email_notifier.py` (Brevo) belong to the Application tier; the `email_notifications` table belongs to the Data tier.

Simple example (a restaurant): the menu and waiter are the presentation tier, the kitchen is the application tier, and the store room is the data tier. The customer never enters the store room; the waiter never cooks. Because each part is separate, you can redesign the menu (GUI) without touching the kitchen (AI code).

PART K: FRONTEND TOOLS, WIDGETS AND ELEMENTS (added from the report)

The interface is built with native Tkinter plus Pillow, and a custom animation drawn on a Canvas. No web browser is needed.

Tool / widget	Where used	What it does (simple)
`tk.Tk` / `tk.Toplevel`	`main.py`, all sub-windows	One main window (root) and child windows; keeps a single-root application
`tk.Canvas`	`login.py`, `dashboard.py`	Draws the animated 34-node face mesh on login; shows the wallpaper on the dashboard
`ttk.Treeview` + `Scrollbar`	`student.py`, `attendance.py`	Table of students / attendance with column headings and scrolling
`ttk.Combobox`	`student.py`	Drop-down lists: Department (**35 options in the code**; the report says "29+"), Course, Year (up to 2030-2031), Semester
`tk.LabelFrame` / `Frame`	`student.py`, `login.py`	Groups related fields inside a titled border (`relief=tk.GROOVE`, `bd=2`)
`tk.Entry` / `tk.Button`	All modules	Text boxes (password hidden with `show='*'`) and action buttons
PIL / Pillow (`Image.Resampling.LANCZOS`)	`dashboard.py`, `developer.py`	Loads and resizes pictures smoothly without pixelation
Digital clock	`dashboard.py`, `student.py`	Label updated every 1000 ms with `self.root.after(1000, ...)`

Definitions with examples

- **LANCZOS:** a high-quality resizing method. Example: shrinking a 1366-px banner to fit the window while keeping it sharp.
- **Canvas:** a drawing board inside the window. Example: the login screen draws 34 dots and lines on it, then redraws every 35 ms to make them move.
- **Responsive layout:** the window adapts to the screen size. Example: `win_w = min(1280, int(screen_w * 0.92))` never exceeds the screen.

> Note: in the code the login animation redraws every 35 ms, which is about **28 frames per second**, not 60.

PART L: BACKEND AND AI TOOLS (updated to the final version)

- **Python 3.10+:** runs the logic, threads and error handling.
- `cv2.VideoCapture` (`CAP_DSHOW` on Windows, with fallback): reads webcam frames in BGR colour.
- **Haar Cascade** (`haarcascade_frontalface_default.xml`, about 930 KB): finds frontal faces. Capture uses `1.1 / 4 / 40x40`; live detection uses `1.15 / 6`.
- **2x downsampling** (`fx=0.5, fy=0.5`): 75% fewer pixels, faster detection.
- **Shape filter:** keep boxes with width/height between 0.70 and 1.35.
- **LBPH recognizer:** radius 2, neighbours 8, grid 8x8, faces 200x200. The technology-stack PDF said radius 1; **the code and the final report say 2**.
- **Distance and threshold:** lower is closer; 70 / **78** / 82; 50 or less is accepted at once.
- **Tracker and voting:** last 6 predictions per face.
- **Liveness module:** 2 random actions, 5.5 s each, 12 s total, fails closed.
- **Training hub:** reads `user.<id>.<n>.jpg`, uses only enrolled students, saves `classifier.xml` and `id_map.json`.
- **Security module:** PBKDF2-HMAC-SHA256 with salt.
- **E-mail module:** Brevo API in a background thread.
- **Session cache:** `Remember Me` keeps only the username.

Definitions with examples

- **Viola-Jones:** the 2001 face-detection method (Haar features, integral image, AdaBoost, cascade).
- **Integral image:** a precomputed table that lets the computer add up any rectangle's pixels in a constant number of steps (like a running total in a spreadsheet).
- **O(1) time:** time does not grow with the size of the rectangle.
- **Downsampling:** 640x480 becomes 320x240, a quarter of the pixels.
- **Chi-square distance:** a number showing how different two histograms are; 0 is identical.
- **Serialization:** saving a trained model to a file (`classifier.xml`).

PART M: DATABASE AND PERSISTENCE LAYER IN DETAIL (updated)

Item	Source	Status
SQLite 3, file-based, serverless	Final report, PPT	Confirmed
WAL mode (recognition can write while another window reads)	Final report, PPT	Confirmed
4 tables (`users`, `students`, `attendance`, `email_notifications`)	Final report, PPT	Confirmed
Unique index on `attendance(student_id, date)`	Final report, PPT, text file	Confirmed
Unique `email_notifications(student, date, status)`	Final report, PPT	Confirmed
Hashed passwords and security answers	Final report, PPT	Confirmed
Bound (`?`) parameters	Final report	Confirmed
`PRAGMA busy_timeout = 60000`	Technology-stack PDF only	**Not confirmed**; check `db_manager.py`
CSV synchronisation (after a successful DB commit)	Final report	Confirmed

Dual persistence: every attendance event goes to the **database** (safe, structured) and then to the **CSV** (easy for teachers to open in Excel).

```python
# ILLUSTRATIVE SKETCH
conn = sqlite3.connect("database/attendance_system.db")
conn.execute("PRAGMA journal_mode = WAL")
try:
    conn.execute("INSERT INTO attendance (student_id, roll, name, department, t
                 "VALUES (?, ?, ?, ?, ?, ?, ?)", row)       # fails if (student
    conn.commit()
    write_csv_row(row)                                       # only after commi
except sqlite3.IntegrityError:
    pass                                                     # already marked t
```

Definitions with examples

- **ACID:** Atomic (all or nothing), Consistent, Isolated, Durable. A bank transfer either completes fully or not at all.
- **Transaction:** a group of steps treated as one unit.
- **Journal / WAL:** a log the database writes before changing the real file.
- **Serverless database:** no separate program to run; SQLite is just a file.
- **PRAGMA:** a SQLite command that changes settings.
- **Schema:** the blueprint of a table.
- **Upsert:** insert a row, or update it if it already exists.

PART N: RESULTS AND MEASUREMENTS (replaced with the final report's values)

> The older technology-stack PDF contained benchmark charts (62.4 FPS, 91-98% accuracy, WAL timings). Your **final report states that no accuracy or error rates were measured**, so those chart numbers should **not** be quoted. Use the final report's table below.

N1. Measurements reported from one development laptop (Intel Core i5, Windows 11, 720p webcam; rough values)

Measurement	Value
Face detection per frame	11.8 to 13.2 ms (half-size grayscale)
LBPH matching per face	3.8 to 4.9 ms
Liveness overhead	under 1.5 ms per frame
Live video rate	52 to 60 FPS (cannot exceed the camera's own frame rate)
Database write	under 2 ms (WAL)
Blink window / time limits	0.10 to 1.20 s; 5.5 s per action; 12 s in total

N2. LBPH distance ranges (lower = closer)

Case	Distance
Enrolled, bright light	41.2 to 55.4 (average 46.8)
Enrolled, ambient light	66.5 to 73.8 (average 69.5)
Unregistered people	81.4 to 89.2
Non-face background	above 92

At the default threshold **78**, enrolled students are accepted and unregistered people are rejected **in these small tests.** At 70 some ambient-light students would be rejected; 82 reaches into the

unregistered range.

N3. Development copy: 6 students, 615 images (100 to 106 each), 51 attendance rows, 2 notification rows, model about 77.5 MB.

N4. Scale calculation (be ready to show it)

- One histogram = 64 cells x 256 bins = **16,384 values**; at 8 bytes each = 131,072 bytes (about 128 KiB). For 615 images that is about 77 MB, matching the reported 77.5 MB.
- About 126 KB per image, so 100 images give **about 12.6 MB per student**.
- 300 students = 30,000 images, so about **3.8 GB**.
- Matching time grows in proportion: about 4.35 ms (middle of 3.8 to 4.9) x (30,000 / 615 = 48.8) = **about 0.2 s per face**.

How to explain in the viva: "These are rough values from one laptop. We did not record how many unregistered people were tried and did not compute false-accept or false-reject rates, so the thresholds still need validation on a larger group."

PART O: WHY THESE TECHNOLOGIES WERE CHOSEN (TRADE-OFF ANALYSIS) (added from the report)

Area	Chosen	Alternative considered	Why chosen	Why alternative rejected
Face recognition	**LBPH**	Deep CNN (FaceNet/ResNet)	About 3.8 to 4.9 ms per face (reported), runs on CPU, handles lighting changes, trains instantly	Needs a strong GPU; about 85 ms per face makes live video very slow
Face recognition	**LBPH**	Eigenfaces / Fisherfaces	Looks at small local neighbourhoods, so shadows matter less	Use global pixels; accuracy falls under shadows
Face detection	**Haar Cascade (2x downsample)**	YOLOv8 / SSD MobileNet	Fast on CPU, tiny 930 KB XML, no deep-learning dependency	Needs heavy PyTorch (800 MB+), high CPU use
Database	**SQLite with WAL**	MySQL / PostgreSQL	No server, one file, ACID, very low latency	Needs a background server and network/port setup
Persistence	**Database + CSV**	Database only	Teachers can open the CSV in Excel; database keeps structure	Teachers can't easily open SQLite files
Frontend	**Tkinter + PIL**	Electron (HTML/CSS/JS)	Starts in under a second, very little RAM, native windows	Electron needs a heavy Chromium runtime (200+ MB RAM)
Login animation	**Canvas vector drawing**	Embedded MP4/GIF	Low CPU, no codecs, scales cleanly	Video decoding uses more CPU, needs FFmpeg, looks blurry when scaled

Simple way to say it: "We chose the lightest tool that does the job well on an ordinary college computer."

Honest limitation to mention: LBPH is less accurate than deep-learning recognition when there are many students, or large changes in angle, expression or age. The project trades some accuracy for speed and simplicity.

PART P: HARDWARE AND SOFTWARE REQUIREMENTS (added from the report)

Resource	Minimum	Recommended	Why needed
Operating system	Windows 10/11 (64-bit) (the code is written mainly for Windows)	Windows 11 (64-bit)	Runs the app and camera
Python	3.10+	3.11 / 3.12 (virtual environment)	Runs the code
CPU	Intel Core i3 (4th gen) / Ryzen 3	Core i5 (8th gen+) / Ryzen 5	Face detection and recognition
RAM	4 GB	8 GB+	Holds frames and GUI
Camera	Laptop webcam (720p)	External USB 1080p webcam (30+ FPS)	Captures students
Disk	500 MB free	2 GB free (SSD)	Photos (about 100 per student), database, CSV

Libraries: `opencv-contrib-python>=4.8.0`, `pillow>=10.0.0`, `numpy>=1.24.0`, `pandas>=2.0.0`, `mysql-connector-python>=8.0.0`.

PART Q: TECHNICAL SUMMARY (for your conclusion)

FaceAttend Ultra shows that a face-recognition attendance system can be built from a Haar cascade, an LBPH recognizer and an SQLite database on an ordinary laptop. Consistent image preparation, a distance threshold and voting over six frames reduce wrong matches, and a random liveness challenge that fails closed adds a further check before attendance is recorded. Attendance is stored once per student per day, credentials are stored as salted hashes, training uses only enrolled students, and an optional parent e-mail is sent through Brevo. It is a low-cost prototype: testing against real photo and video attacks and a controlled accuracy evaluation are still to be done.

PART R: ADDITIONAL GLOSSARY (terms from the report)

Term	Simple definition	Example
3-tier architecture	Splitting an app into presentation, logic and data layers	Restaurant: menu, kitchen, store room
Separation of concerns	Each part does one job	GUI code doesn't contain SQL
FPS	Frames per second	30 FPS = 30 pictures each second
Latency	Time one operation takes	3.8 to 4.9 ms to recognise a face
Eigenfaces (PCA)	Older recognition using the whole image's main patterns	Sensitive to shadows
Fisherfaces (LDA)	Eigenfaces improved to separate people better	Needs more data
CNN / FaceNet	Deep neural network for faces	High accuracy, needs a GPU
YOLO / SSD	Deep-learning object detectors	Heavy for a simple face box
Electron	Framework that builds desktop apps from web pages	Bundles a whole browser
Serverless database	A database with no separate server program	SQLite
SQL injection	Attack by typing SQL into an input box	Stopped by `?` parameters
Illumination (lux)	Brightness of light	400-600 lx = normal classroom
Occlusion	Part of the face covered	Glasses, hair, mask
Bounding box	Rectangle around a detected face	`(x, y, w, h)`
Telemetry / benchmark	Measured performance data	The FPS bar chart

---

PART S: EXTRA VIVA QUESTIONS

Merged into Part G above (Q1 to Q60), so there is one question bank.

PART T: CONSISTENCY CHECK: sources compared (updated)

Item	Technology-stack PDF	Earlier uploaded code	Final report / PPT	Use in viva
Default threshold	`< 75`	`<= 82`	**78** (70 / 78 / 82 selectable)	78
LBPH radius	1	2	**2**	2
Capture Haar settings	n/a	1.3 / 5 / 60x60	**1.1 / 4 / 40x40**	final report
Live Haar settings	n/a	1.12 / 4 / 25x25	**1.15 / 6**	final report
Decision	single frame	single frame	**6-frame voting + liveness**	final report
Remember Me	"login tokens"	username + password	**username only**	username only
Passwords	"hashed"	unverified	**PBKDF2-HMAC-SHA256 + salt**	hashed
Recovery wording	"2FA"	security question	**security-question recovery**	not 2FA
Video rate	62.4 FPS	n/a	**52 to 60 FPS, camera-limited**	52 to 60
Accuracy	91-98% charts	n/a	**not measured**	do not quote
LBPH time per face	4.2 ms	n/a	**3.8 to 4.9 ms**	range
Login animation	60 FPS	`after(35)` is about 28 FPS	n/a	about 28 FPS
Students columns	15	15	**16 (adds `parent_email`)**	16
Tables	3	3	**4 (adds `email_notifications`)**	4
Landmark localisation, pandas for CSV, Linux/macOS	claimed	not in code	not claimed	do not claim

PART U: COMPLETE 12-STEP WORKFLOW (fills the gaps in your text files)

Your text files jump from Step 5 to Step 8, have no step for the liveness challenge, and have no step for review and export. The complete flow is below; the last column shows where each step appears in your text.

Step	What happens	Key facts	In your text
1	Administrator login	DB and folders prepared, login window, hashed check, dashboard	Part 2, Step 1
2	Student enrolment	Student ID, department, course, year, semester, division, name, roll, gender, DOB, e-mail, phone, address, mentor, **parent e-mail**	Step 2
3	Face sample capture	Webcam, Haar, crop, grayscale, 200x200, equalise, save in `data/`, about 100 images	Step 3
4	Model training	Read images, take IDs from file names, keep only enrolled students, integer labels, LBPH, save model	Step 4
5	Live face detection	Frame to grayscale, half size, Haar, shape filter 0.70 to 1.35, scale back	Step 5
6	**Recognition (missing)**	Same preparation as training; LBPH returns label + distance (lower = closer)	add
7	**Threshold (missing)**	70 / 78 / 82; distance 50 or less accepted at once	add
8	Multi-frame voting	Last 6 predictions per tracked face	Step 8
9	**Liveness challenge (missing)**	2 random actions, 5.5 s each, 12 s total, fails closed	add
10	Attendance recording	SQLite once per day (unique index), then CSV	Step 9
11	Parent e-mail	Background thread, Brevo, audit row	Step 10
12	**Review, edit, import, export (missing)**	Attendance window, CSV	add

Corrections to make in your text files

- Replace the stray line `200 * 200 = 40,000 text{ pixels}` with `200 x 200 = 40,000 pixels` (a formatting code was left in).
- Remove the stray `15.` before "Step 4 - Model Training".

- Renumber the steps so none are missing, using the table above.
- "Your project / your report" wording: change to "our project / this report" if the text is copied into the submission.

PART V: ADDITIONAL GLOSSARY (final-version terms)

Term	Simple definition	Example
Liveness detection	Checking a live person is present, not a photo or video	Blink on request
Spoofing	Trying to fool the system	Holding up a photo
Replay attack	Playing a recorded video of the real person	Video on a phone
Injection attack	Feeding fake video straight into the system	Virtual camera
Fail closed	When unsure, deny	Face lost, so no attendance
Baseline	Short measurement of the face before actions	Resting head position
Cooldown	Short wait before retrying	After a failed check
Multi-frame voting	Decide from several frames	5 of 6 say "Rahul"
Tracker	Follows the same face across frames	Keeps a face's last 6 predictions
Aspect ratio	Width divided by height	100 / 110 = 0.91
Hash	One-way scrambled version of data	SHA-256 digest
Salt	Random data mixed in before hashing	Same password, different hash
PBKDF2	Password hashing that repeats work to slow guessing	Thousands of repeats
HMAC	Hash combined with a secret key	Message authentication
Constant-time comparison	Comparison that takes the same time whatever the input	Stops timing leaks
Bound parameter	Value passed separately from SQL text	`WHERE email = ?`
SQL injection	Attack by typing SQL into an input	Blocked by `?`
Unique index	Database rule forbidding repeats	student_id + date
Primary key	Column that uniquely identifies a row	`student_id`

Term	Simple definition	Example
Cascade deletion	Deleting a record also deletes related rows	Delete student, delete linked rows
API / REST	A way for programs to talk over the internet	Brevo's e-mail API
Transactional e-mail	Automatic e-mail triggered by an event	Attendance notice
Audit record	Log of what happened and the result	`email_notifications` row
Upsert	Insert or update	Update a notification row
Background thread	Work done in parallel	E-mail sending
Mock	A fake stand-in used in tests	Fake Brevo service
Synthetic data	Made-up test data	Artificial frames
FAR / FRR	False-accept / false-reject rate	Not measured yet
Embedding model	Deep model that turns a face into a vector	ArcFace-style
Consent and retention	Permission to store data, and how long	Delete photos after the course
Prototype	Working early version, not finished product	This project

---

PART W: REVIEW OF YOUR FINAL REPORT, PPT AND TEXT FILES

What is good: the numbers agree across the report, PPT and text files (6 students, 615 images, 51 rows, 38 tests, 77.5 MB, thresholds 70/78/82). The limitations are honest and specific, which examiners usually respect.

Fix or add in the report

1. Make the wording consistent: the cover says "A Project Report", the certificate says "mini project". Use your department's exact term (for example "Minor Project").
2. Table 3, row "Distance, unregistered people": the comment "Number of people not recorded" is unclear. Reword it as "the number of people tested was not recorded".
3. Add the missing items listed in Part I (list of figures and tables, abbreviations, requirements, diagrams, screenshots).
4. Add the simple LBP example (Q17) and the Haar explanation to Methodology so an examiner sees you understand the algorithms.
5. "Individual contribution to report preparation / presentation" is the same sentence for everyone. Make it specific per member.

6. Do **not** import the old technology-stack PDF's benchmark numbers (see Part N).

PPT (10 slides)

1. Add a **screenshots / live demo slide** (login, enrolment, recognition with the liveness prompt, attendance table).
2. Add a **Questions** slide, and plan who presents which slide (each member explains their own module, as the report says).
3. The title slide lists names in a different order from the report; use the same order.
4. Slide 3 (tables) is dense; say the four table names aloud and show only the key fields.
5. The speaker notes are good; practise them aloud.

Text files: see the corrections at the end of Part U.

One example to explain the whole project

> A **guard** (Haar Cascade) notices that a face has appeared and draws a box around it. A **memory expert** (LBPH) studies the tiny texture patterns of that face, turns them into a signature, and compares it with the signatures memorised during **Train Data**. If the difference is **82 or less**, the expert announces "This is student S101". The system then writes the student's name, date and time into the **database** and the **CSV register**, only once a day. The **Attendance** screen lets the teacher correct, import and export that register, and the **login** keeps outsiders away.